

Grundlagen der C-Programmierung

Dr. Henrik Brosenne
Georg-August Universität Göttingen
Institut für Informatik

Wintersemester 2024/25

Steuerung des Programmablaufs

Anweisungen und Blöcke

Die `if`-Anweisung

Einschub: Aufzählungstyp `enum`

Die `switch`-Anweisung

Die `while`-Schleife

Die `do`-Schleife

Die `for`-Schleife

Anweisungen und Blöcke

Die einfachsten C-Anweisungen sind die **Ausdrucksanweisungen**. Sie bestehen aus einem Ausdruck, dem ein Semikolon folgt.

Beispiele

```
a = b + c;  
++i;
```

Das Semikolon ist in C ein **Abschlusssymbol** und kein **Trennsymbol**.

Mehrere Anweisungen können zu einem **Block** zusammengefasst werden, indem man sie in ein Paar **geschweifte Klammern** (`{ }`) einschließt.

Achtung: Nach der geschweiften Klammer, die einen Block abschließt, steht **kein** Semikolon.

Ein Block ist syntaktisch einer einzelnen Anweisung äquivalent.

Variablen, die **innerhalb eines Blocks deklariert** werden sind auch nur **innerhalb dieses Blocks gültig**. Gilt auch z.B. für den Block einer Schleife.

Beispiel, Funktion `main`.

Leere Anweisung

Ein Block darf auch leer sein, d.h. zwischen seinen Klammern brauchen keine Anweisungen zu stehen. Er ergibt dann eine **leere Anweisung**, d.h. eine Anweisung, die nichts bewirkt.

Eine leere Anweisung kann man auch durch aufeinanderfolgende Semikola oder ein Semikolon als ersten Eintrag in einem Block erzeugen.

Beispiele

```
while (getchar () != '\n') {  
}
```

```
while (getchar () != '\n')  
;
```

Steuerung des Programmablaufs

Anweisungen und Blöcke

Die **if**-Anweisung

Einschub: Aufzählungstyp `enum`

Die `switch`-Anweisung

Die `while`-Schleife

Die `do`-Schleife

Die `for`-Schleife

Die if-Anweisung

Vollständige Syntax der `if`-Anweisung.

```
if (expression)
    statement-if
[ else
    statement-else ]
```

Formal wird hinter dem `if` selbst und, wenn vorhanden, auch hinter dem `else` **genau eine** Anweisung verlangt.

Erinnerung. Ein Block ist syntaktisch einer einzelnen Anweisung äquivalent.

Geschachtelten if-Anweisungen (1/2)

Das Schlüsselwort `else` (samt nachfolgender Anweisung) wegzulassen, kann bei geschachtelten `if`-Anweisungen zu Problemen führen.

Beispiel

```
if (n > 0)
    if (n % 2)
        printf("positive and odd\n");
    else
        printf("not positive\n");
```

Es wird folgender Ablauf suggeriert.

- Wenn `n` positiv und ungerade ist, schreibe entsprechende Meldung.
- Wenn `n` positiv und gerade ist, passiert nichts.
- Wenn `n` nicht positiv ist, schreibe entsprechende Meldung.

Tatsächlich ist der Ablauf anders.

- Wenn `n` positiv und ungerade ist, schreibe entsprechende Meldung (wie oben).
- Wenn `n` positiv und gerade ist, wird die Meldung “not positive” geschrieben.
- Wenn `n` nicht positiv ist, passiert nichts.

Geschachtelten if-Anweisungen (2/2)

Den Compiler interessiert es nicht, wie ein Quelltext aufschreibt ist.

Das Einrücken dient nur dazu, dem Leser den Überblick über die Struktur eines Quelltextes zu erleichtern.

Die Zuordnung eines `else` zu einem `if` durch den Compiler erfolgt so, wie die Zuordnung einer schließenden zu einer öffnenden Klammer. Wird eine schließende Klammer gefunden, wird rückwärts gesucht, bis die letzte, passende öffnende Klammer gefunden wird, der noch keine schließende Klammer zugeordnet ist.

Reparaturmöglichkeiten (1/2)

Möglichkeit. Vervollständigen der geschachtelte `if`-Anweisung durch einen `else`-Zweig.

```
if (n > 0)
    if (n % 2)
        printf("positive and odd\n");
    else
        ;
else
    printf("not positive\n");
```

Möglichkeit. Die geschachtelte `if`-Anweisung wird zu einem Block.

```
if (n > 0) {
    if (n % 2)
        printf("positive and odd\n");
}
else
    printf("not positive\n");
```

Reparaturmöglichkeiten (2/2)

Möglichkeit. Umdrehen der Logik.

```
if (n <= 0)
    printf ("not positive\n");
else
    if (n % 2)
        printf("positive and odd\n");
```

Dieses Beispiel legt eine etwas veränderte Schreibweise nahe.

Wenn die Anweisung hinter einem `else` ihrerseits eine `if`-Anweisung ist, liegt es nahe, beide Zeilen zu einer zusammenzufassen.

```
if (n <= 0)
    printf("not positive\n");
else if (n % 2)
    printf("positive and odd\n");
```

Die Form, in der man seine Programme aufschreibt, sollte schon auf den ersten Blick einen möglichst guten Überblick über die Struktur der Programme erlauben. Dazu dient unter anderem das Einrücken.

Beispiel. Berechnung von Kreisumfang oder Fläche (1/2)

```
1  /*****
2   * circumference or area (if/else)
3   *****/
4  #include <stdio.h>
5
6  #define PI 3.14159265359
7
8  int main(void) {
9      int ch;
10     double radius;
11
12     printf("Circle radius: ");
13     scanf("%lf", &radius);
14     while (getchar () != '\n')
15         ;
16
```

Beispiel. Berechnung von Kreisumfang oder Fläche (2/2)

```
17     printf("Computation of circumference (c/C) or area (a/A)? ");
18
19     ch = getchar();
20     if (ch == 'c' || ch == 'C')
21         printf("Circumference: %E\n", 2*PI*radius);
22     else if (ch == 'a' || ch == 'A')
23         printf("Area: %E\n", PI*radius*radius);
24     else
25         printf("Illegal selection\n");
26
27     return 0;
28 }
```

Steuerung des Programmablaufs

Anweisungen und Blöcke

Die `if`-Anweisung

Einschub: Aufzählungstyp `enum`

Die `switch`-Anweisung

Die `while`-Schleife

Die `do`-Schleife

Die `for`-Schleife

Aufzählungstyp enum

Aufzählungskonstanten sind Konstanten mit ganzzahligem Wert, die durch eine `enum`-Deklaration einen **Aufzählungstyp** erhalten, mit dem sie im weiteren angesprochen werden können.

```
enum [name] { enumerator-list };
```

Dabei:

- Das Schlüsselwort `enum` leitet die Deklaration ein.
- Der Aufzählungstyp benannt werden. Die eckigen Klammern bedeuten, dass der Name *name* optional ist.
- Die Liste der Konstanten wird in geschweifte Klammern eingeschlossen. Die Elemente der Liste werden voneinander jeweils durch ein Komma getrennt. Es ist üblich, wenn auch nicht notwendig, für die Namen der Konstanten ausschließlich Großbuchstaben zu verwenden, was der Lesbarkeit des Programms dienen soll.
- Die Vereinbarung wird durch ein Semikolon abgeschlossen.

Beispiel (1/2)

Deklaration

```
enum color { RED, YELLOW, BLUE, GREEN };
```

Dem ersten Namen in einer `enum`-Liste wird der Wert 0 zugeordnet, allen weiteren der um eins erhöhte Wert des Vorgängers.

Man kann die Werte jedoch auch explizit in der Form `= value` angeben. Dann wird für Namen, für die kein Wert angegeben ist, aufsteigend weitergezählt.

```
enum escapes { BELL = '\a' , NEWLINE = '\n' };  
enum color { RED = 1, YELLOW = 4, BLUE, GREEN = 2 };
```

Hier erhält `BLUE` den Wert 5, während allen anderen Namen die angegebenen Werte zugeordnet werden.

Beispiel (2/2)

Definition

```
enum color { RED, YELLOW, BLUE, GREEN };
```

Definition einer Variablen mit Initialisierung

```
enum color paint = YELLOW;
```

Benutzung

```
if (paint == YELLOW) {  
    ...  
}
```


Alle Namen in einer oder **verschiedenen** Aufzählungsdeklarationen im selben Gültigkeitsbereich müssen sich unterscheiden.

Die Werte, die verschiedenen Namen zugeordnet werden, auch innerhalb einer einzelnen Aufzählungsdeklaration, brauchen nicht verschieden zu sein.

Auch negative Werte können vergeben werden.

Die Werte müssen **ganzzahlig** sein.

Steuerung des Programmablaufs

Anweisungen und Blöcke

Die `if`-Anweisung

Einschub: Aufzählungstyp `enum`

Die **`switch`**-Anweisung

Die `while`-Schleife

Die `do`-Schleife

Die `for`-Schleife

Die switch-Anweisung

Oft soll, in Abhängig von einem bestimmten Wert, nicht nur aus zwei, sondern aus mehr Möglichkeiten ausgewählt werden.

C kennt speziell dafür die **switch**-Anweisung

```
switch (expression) {  
    case value-1:  
        statement-sequence-1  
    case value-2:  
        statement-sequence-2  
    ...  
    case value-N:  
        statement-sequence-N  
[ default:  
    statement-sequence-D ]  
}
```

Ablauf.

- Der Ausdruck *expression* wird ausgewertet.
- Ist das Resultat einer der einem **case** folgenden Werte *value-1*, ..., *value-N*, wird die Ausführung mit der Anweisungsfolge, die dem Wert folgt, fortgesetzt.
- Kommt der Wert des Ausdrucks nicht vor, wird die Anweisungsfolge hinter **default** ausgeführt, wenn eine **default**-Klausel angegeben ist, bzw. sonst direkt hinter das Ende der **switch**-Anweisung verzweigt.

Formalien

- Sowohl der Ausdruck *expression* als auch die Werte *value-1*, ..., *value-N* müssen ganzzahlig sein. Gleitkommazahlen sind nicht erlaubt.
- Die Werte *value-1*, ..., *value-N* müssen sämtlich verschieden sein.
- Die Werte *value-1*, ..., *value-N* müssen Konstanten oder **konstante Ausdrücke** sein. Unter einem konstanten Ausdruck versteht man einen Ausdruck, dessen Operanden sämtlich Konstanten sind, in dem keine Funktionen aufgerufen werden und in dem keine Operatoren mit Nebeneffekten vorkommen.

Hintergrund. Konstante Ausdrücke werden bereits durch den Compiler ausgewertet, sind bei der Ausführung des Programms also gleichzusetzen mit Konstanten.

- Die Reihenfolge, in der *value1*-, ..., *value-N* und ggf. **default** angegeben werden, ist beliebig. Bei der Reihenfolge, die man wählt, sollte man die Lesbarkeit des Programms allerdings besonders im Auge haben.
- Jeder **case**- und der **default**-Klausel darf eine **Anweisungsfolge** folgen, nicht nur eine einzelne Anweisung. Insbesondere entfällt damit die Notwendigkeit, Anweisungsfolgen durch geschweifte Klammern zu Blöcken zusammenzufassen.

Beispiel (1/2)

Ein Programm, das mit Wochentagen arbeitet, wird die Wochentage mit Kennziffern identifizieren, etwa Sonnabend mit 0, Sonntag mit 1, usw..

Will man diese Kennziffern in Klarschrift umsetzen, so kann man das mit einer **switch**-Anweisung tun (auch wenn es gerade für dieses Beispiel günstigere Möglichkeiten gibt).

```
enum weekday {MONDAY, TUESDAY, ..., SUNDAY};  
...  
enum weekday day;  
...  
switch (day) {  
    case MONDAY:  
        printf("monday\n");  
    case TUESDAY:  
        printf("tuesday\n");  
    ...  
    case SUNDAY:  
        printf("sunday\n");  
    default:  
        printf("incorrect weekday code\n");  
}
```

Beispiel (2/2)

Das Beispiel funktioniert für unzulässige Kennzahlen famos – und liefert für zulässige Kennzahlen nicht das beabsichtigte Ergebnis. Ist die Kennzahl etwa FRIDAY, so erhält man drei Ausgabezeilen

```
friday
saturday
sunday
incorrect weekday code
```

Was geht schief?

Wenn die Anweisungsfolge für einen Fall ausgeführt ist, wird nicht automatisch hinter das Ende der **switch**-Anweisung verzweigt, sondern linear mit den Anweisungen für die nachfolgenden Fälle fortgefahren.

Abhilfe schafft die Anweisung **break**. Ihre Ausführung bewirkt, dass die Bearbeitung der **switch**-Anweisung sofort beendet und mit der ihr folgenden Anweisung fortgefahren wird.

Korrigiertes Beispiel

```
enum weekday {MONDAY, TUESDAY, ..., SUNDAY};  
...  
enum weekday day;  
...  
switch (day) {  
    case MONDAY:  
        printf("monday\n");  
        break;  
    case TUESDAY:  
        printf("tuesday\n");  
        break;  
    ...  
    case SUNDAY:  
        printf("sunday\n");  
        break;  
    default:  
        printf("incorrect weekday code\n");  
}
```

Ob man hinter der letzten Klausel eine **break**-Anweisung schreibt oder nicht, ist reine Geschmackssache, sie hat auf den Programmablauf keinen Einfluss.

Beispiel. Kreisumfang oder Fläche mit switch (1/2)

```
1  /*****
2   * circumference or area (switch)
3   *****/
4  #include <stdio.h>
5
6  #define PI 3.14159265359
7
8  int main() {
9      int ch;
10     double radius;
11
12     printf("Circle radius: ");
13     scanf("%lf", &radius);
14     while (getchar () != '\n')
15         ;
16
```

Beispiel. Kreisumfang oder Fläche mit switch (2/2)

```
17     printf("Computation of circumference (c/C) or area (a/A)? ");
18
19     ch = getchar();
20     switch(ch) {
21     case 'c':
22     case 'C':
23         printf("Circumference: %E\n", 2*PI*radius);
24         break;
25     case 'a':
26     case 'A':
27         printf("Area: %E\n", PI*radius*radius);
28         break;
29     default:
30         printf("Illegal selection\n");
31     }
32
33     return 0;
34 }
```

Steuerung des Programmablaufs

Anweisungen und Blöcke

Die `if`-Anweisung

Einschub: Aufzählungstyp `enum`

Die `switch`-Anweisung

Die **`while`**-Schleife

Die `do`-Schleife

Die `for`-Schleife

Die while-Schleife

Die **while**-Schleife wurde schon eingeführt.

```
while (expression)
    statement
```

Solange der Ausdruck *expression* wahr – d.h. ungleich Null – ist, wird die folgende Anweisung ausgeführt.

Anmerkung. Wie bei der **if**-Anweisung besteht formal der Rumpf der Schleife aus genau einer Anweisung. Diese kann ein Block sein, muss es aber nicht.

Steuerung des Programmablaufs

Anweisungen und Blöcke

Die `if`-Anweisung

Einschub: Aufzählungstyp `enum`

Die `switch`-Anweisung

Die `while`-Schleife

Die `do`-Schleife

Die `for`-Schleife

Die do-Schleife

Die zweite Schleifenanweisung ist die **do**-Anweisung.

```
do
    statement
while (expression);
```

Formatierung mit Block, um die Verwechslung der **while**-Klausel mit einer eigenständigen **while**-Anweisung zu verhindern.

```
do {
    statement
} while (expression);
```

do- und while-Schleife (1/2)

do- und while-Schleife sind sich sehr ähnlich.

Der Unterschied zwischen den beiden Schleifen ist

- bei einer do-Schleife wird sofort der Rumpf ausgeführt und erst danach die Bedingung überprüft wird,
- bei einer while-Schleife ist die erste Operation eine Überprüfung der Bedingung ist

Beide Schleifenanweisungen lassen sich ohne weiteres durcheinander ersetzen.

```
do
    statement
while (expression);
```

entspricht

```
statement
while (expression)
    statement
```

Die Formulierung als do-Schleife wirkt *natürlicher* und spart Quelltextzeilen, insbesondere wenn *statement* ein Block ist.

do- und while-Schleife (2/2)

Umgekehrt

```
while (expression)
    statement
```

entspricht

```
if (expression)
    do
        statement
    while (expression);
```

Hier wirkt die Formulierung als `while`-Schleife *natürlicher* und spart Quelltextzeilen.

Im Einzelfall sollte man diejenige der beiden Schleifenanweisungen verwenden, die die *natürlichere*/kürzere Formulierung erlaubt.

Anwendung der do-Schleife

In der Praxis kommen **while**-Schleifen häufiger als **do**-Schleifen vor, weil man in der Regel zunächst zu prüfen hat, ob der Schleifenrumpf überhaupt ausgeführt werden darf, bevor man ihn ausführt.

Das Kopieren eines String ist eine Ausnahme, weil ein Zeichen, nämlich das abschließende Null-Zeichen, auf jeden Fall kopiert werden muss.

Ein typisches Beispiel für **do**-Schleifen ist auch die Kommunikation mit dem Benutzer. Zunächst muss er seine Eingabe vornehmen – gibt er Unsinn ein, muss die Abfrage nach einer Ermahnung wiederholt werden.

Auswahl aus Alternativen (neu)

```
1  #include <stdio.h>
2
3  int main() {
4      int value;                                // no init
5
6      do {
7          printf("Enter a positiv value: ");
8          if (scanf("%d", &value) != 1) {        // read value
9              printf("Input error\n");          // error handling
10             return 9;
11         }
12         if (value <= 0)                         // input ok?
13             printf("Value not positiv - try again\n");
14
15     } while (value <= 0);
16
17     printf("ok\n");
18
19     return 0;
20 }
```

Steuerung des Programmablaufs

Anweisungen und Blöcke

Die `if`-Anweisung

Einschub: Aufzählungstyp `enum`

Die `switch`-Anweisung

Die `while`-Schleife

Die `do`-Schleife

Die `for`-Schleife

Die for-Schleife

Die dritte Schleifenanweisung in C ist die *for*-Anweisung

```
for (clause-1; expression-2; expression-3)
    statement
```

Der Ablauf lässt sich am einfachsten durch eine *while*-Schleife beschreiben.

```
clause-1;
while (expression-2) {
    statement
    expression-3;
}
```

Dabei kann *clause-1* eine Deklaration oder ein Ausdruck sein.

Der **Wert** von *clause-1*, wenn es ein Ausdruck ist, und/oder von *expression-3* wird nicht verwendet. Man nutzt nur die Nebeneffekte, die bei der Berechnung der Ausdrücke auftreten, z.B. beim Auswerten des Zuweisungs- oder Inkrementierungsoperators.

- *clause-1* dient (normalerweise) der Initialisierung,
- *expression-3* dient (normalerweise) der Aktualisierung der Schleifenvariablen.

Einsatz von `for`-Schleifen

Es spricht manches dafür, sich freiwillig an folgende Unterscheidung zu halten.

- `for`-Schleifen werden eingesetzt, wenn die Anzahl der Schleifendurchläufe bereits beim Eintritt in die Schleife bekannt ist
- Wenn die Wiederholungsbedingung im Rumpf der Schleife modifiziert werden soll, verwendet man eine `while`- (`do-while`-)Schleife.

Typische Beispiele für den Einsatz der `for`-Schleife sind in diesem Sinne Operationen auf Feldern.

Arbeit mit Vektoren. Skalarprodukt mit for-Schleife (1/2)

```
1  /*****
2   * scalar product (for loop)
3   *****/
4  #include <stdio.h>
5  #define N 5
6
7  //=====
8  int main() {
9      double v[N], w[N], prod;          // declaration of variables
10     int i;
11
12     printf("Enter first vector\n");    // read 1st vector
13     for (i = 0; i < N; i++)
14         scanf ("%lf", &v[i]);
15
16     printf("Enter second vector\n");   // read 2nd vector
17     for (i = 0; i < N; i++)
18         scanf ("%lf", &w[i]);
19 }
```

Arbeit mit Vektoren. Skalarprodukt mit for-Schleife (2/2)

```
20 // compute scalar product -----
21 prod = 0;
22 for (i = 0; i < N; i++)
23     prod = prod + v[i] * w[i];
24
25 printf("Scalar_\u005Cproduct:_%f\n", prod);
26
27 return 0;
28 }
```

Schrittweite

Die Schrittweite einer `for`-Schleife muss nicht notwendig $+1$ (oder -1) sein.

Beispiel

```
long power;  
  
for (power = 1; power <= 100000; power *= 5)  
    printf("%ld\n", power);
```

Es werden alle Potenzen von 5 ausgegeben, die kleiner oder gleich 100000 sind.

Ausdrücke in for-Schleifen

Die drei Ausdrücke einer **for**-Klausel sind **optional**, können also auch weggelassen werden. Nur die Semikola, die die Ausdrücke trennen, **müssen** geschrieben werden.

Beispiel

```
long power = 1;

for (; power <= 100000; power *= 5)
    printf("%ld\n", power);
```

oder auch

```
long power = 1;

for (; power <= 100000;)
{
    printf("%ld\n", power);
    power *= 5;
}
```

Insbesondere im letzten Beispiel sollte man sich aber überlegen, ob eine Realisierung mit einer **while**-Schleife nicht angemessener wäre.

Möglichkeiten

Eine fehlende Bedingung wird als *wahr* interpretiert.

Beispiel, Endlosschleife

```
for ( ; ; )  
    statement
```

Gelegentlich hat man Schleifen mit zwei (oder mehr) Laufindizes.

Beispiel

Umdrehen der Reihenfolge der Komponenten eines Vektors.

Es ist zweckmäßiger, mit zwei Laufindizes zu arbeiten, als aus Länge und einem Index den anderen Index jeweils neu zu berechnen.

Enthält die Variable *n* die Länge des Vektors, kann man schreiben

```
for (i = 0, j = n - 1; i < j; i++, j--)  
    h = v[i], v[i] = v[j], v[j] = h;
```

Hier zeigt sich, wie man den **Kommaoperator** für kompakte Formulierungen einsetzen kann.

Achtung

Im folgenden Quelltext ist die Klausel der `for`-Schleife

```
int i = 0, x = 2
```

eine Deklaration (mit Initialisierung) der `int`-Variablen `i` und `x`. Deshalb wird in der Schleife nur die `int`-Variable `x` und nicht die `double`-Variable `x` verändert (Stichwort *Verschattung*).

```
1  #include <stdio.h>
2  #define N 10
3
4  int main () {
5      double x = 0;
6      for (int i = 0, x = 2; i < N; i++)
7          x = x * x;
8      printf("%lf\n", x);
9
10     return 0;
11 }
```

Hinweis. Die Klausel kann nur eine Deklaration **oder** eine Anweisung sein.